

PRACTICAL NO. 11

Docker Compose: Building and Orchestrating a Multi-Container Web Application

(Flask + Redis + Nginx Reverse Proxy Stack)

Step 1: Create the project directory structure shown below. Compose expects the docker-compose.yml at the project root; the Flask app and its Dockerfile live in web/, and the Nginx configuration lives in nginx/.



```
Terminal - Project Structure
$ tree practical9
practical9
├── docker-compose.yml
├── web
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
└── nginx
    └── default.conf

3 directories, 5 files
```

Step 2: Create the Flask application file web/app.py. This app increments a counter stored in Redis on every request and returns the current count along with the container's hostname (useful later when we scale the web service).

web/app.py

```
from flask import Flask
from redis import Redis
import socket

app = Flask(__name__)
redis = Redis(host='redis', port=6379, decode_responses=True)

@app.route('/')
def hello():
    count = redis.incr('hits')
    hostname = socket.gethostname()
    return (f'Hello! This page has been viewed {count} times.\n'
            f'Served by container: {hostname}\n')
```

```
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

Step 3: Create web/requirements.txt listing the Python dependencies needed by [app.py](#).

web/requirements.txt

```
flask==3.0.3  
redis==5.0.8
```

Step 4: Create web/Dockerfile to package the Flask application into a custom image.

web/Dockerfile

```
FROM python:3.11-slim  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
COPY . .  
EXPOSE 5000  
CMD ["python", "app.py"]
```

Step 5: Create nginx/default.conf so that Nginx forwards all incoming traffic to the Flask service (using the Compose service name "web" as the hostname).

nginx/default.conf

```
server {  
    listen 80;  
  
    location / {  
        proxy_pass          http://web:5000;  
        proxy_set_header    Host $host;  
        proxy_set_header    X-Real-IP $remote_addr;  
    }  
}
```

Step 6: Create the docker-compose.yml file at the project root. This single file defines all three services, how they are built/pulled, how they are networked, and which ports are published to the host.

docker-compose.yml

```
version: "3.9"
services:
  web:
    build: ./web
    expose:
      - "5000"
    depends_on:
      - redis
    networks:
      - appnet

  redis:
    image: redis:alpine
    networks:
      - appnet

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - web
    networks:
      - appnet

networks:
  appnet:
    driver: bridge
```

Step 7: Build the images defined in the Compose file. Only the "web" service needs building (it has a build: directive); redis and nginx are pulled from Docker Hub.

```
$ docker compose build
```

```
C:\flask-redis-nginx>docker compose build
#1 [internal] load local bake definitions
#1 reading from stdin 513B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 201B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 2.7s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 767B done
#5 DONE 0.0s

#6 [1/5] FROM docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6bab25604534
#6 resolve docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6bab25604534 0.0s done
#6 sha256:d1053354624536b044162aaab1e418bd000ea35184fb1ae098ab3166b1072e72 1.75kB / 1.75kB done
#6 sha256:b8fe4ce3655e95f7f22c2a87d8e03a2f1f0cedc488a8e9adf18cc5a18cfd401 5.49kB / 5.49kB done
```

```
#9 1.876 ----- 101.7/101.7 kB 11.1 MB/s eta 0:00:00
#9 1.883 Downloading redis-5.0-py3-none-any.whl (255 kB)
#9 1.907 ----- 255.6/255.6 kB 12.6 MB/s eta 0:00:00
#9 1.914 Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
#9 1.921 Downloading click-8.5.0-py3-none-any.whl (125 kB)
#9 1.934 ----- 125.3/125.3 kB 11.1 MB/s eta 0:00:00
#9 1.940 Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
#9 1.949 Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
#9 1.962 ----- 134.9/134.9 kB 13.7 MB/s eta 0:00:00
#9 1.969 Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
#9 1.991 ----- 226.5/226.5 kB 10.9 MB/s eta 0:00:00
#9 2.003 Downloading markupsafe-3.0.3-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
#9 2.044 Installing collected packages: redis, MarkupSafe, itsdangerous, click, blinker, Werkzeug, Jinja2, flask
#9 2.402 Successfully installed Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.8 blinker-1.9.0 click-8.5.0 flask-3.0.3 itsdangerous-2.2.0 redis-5.0.8
#9 2.402 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommend
ed to use a virtual environment instead: https://pip.pypa.io/warnings/venv
#9 2.490
#9 2.490 [notice] A new release of pip is available: 24.0 -> 26.2.1
#9 2.490 [notice] To update, run: pip install --upgrade pip
#9 DONE 2.6s

#10 [5/5] COPY . .
#10 DONE 0.0s

#11 exporting to image
#11 exporting layers 0.1s done
#11 writing image sha256:5ef2ff44a1d38dfb7150ccfa6e53d1fa98138eale00c29ea31b9a2c4eb87abba done
#11 naming to docker.io/library/flask-redis-nginx-web done
#11 DONE 0.1s

#12 resolving provenance for metadata file
#12 DONE 0.0s
[+] build 1/1
  Image flask-redis-nginx-web Built 13.0s
C:\flask-redis-nginx>docker compose up -d
[+] up 21/21
  Image redis:alpine Pulled 9.1ss ✓ Image nginx:alpine Pulled 12.1s ✓ N
network flask-redis-nginx_appnet Created 0.0s
  Container flask-redis-nginx-redis-1 Started 0.5s
  Container flask-redis-nginx-web-1 Started 0.5s
  Container flask-redis-nginx-nginx-1 Started 0.8s
```

Step 8: Start the entire stack in detached mode. Compose creates a dedicated bridge network, then starts redis, web, and nginx in the correct dependency order.

```
$ docker compose up -d
```

```
C:\flask-redis-nginx>docker compose up -d
[+] up 21/21
✓ Image redis:alpine Pulled 9.1ss ✓ Image nginx:alpine Pulled 12.1s ✓ N
etwork flask-redis-nginx_appnet Created 0.0s
✓ Container flask-redis-nginx-redis-1 Started 0.5s
✓ Container flask-redis-nginx-web-1 Started 0.5s
✓ Container flask-redis-nginx-nginx-1 Started 0.8s
```

Step 9: Verify that all three containers are running and check their published ports.

```
$ docker compose ps
```

```
C:\flask-redis-nginx>docker compose ps
NAME                IMAGE              STATUS              PORTS
SERVICE            CREATED           STATUS              PORTS
flask-redis-nginx-nginx-1 nginx:alpine       Up 13 seconds       0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
flask-redis-nginx-redis-1 redis:alpine        Up 14 seconds       6379/tcp
flask-redis-nginx-web-1 flask-redis-nginx-web "python app.py"    Up 14 seconds       5000/tcp
web                 15 seconds ago    Up 14 seconds       5000/tcp
```

Step 10: Test the application. Open a browser at <http://localhost:8080> or use curl. Refreshing / repeating the request should increment the hit counter, proving that the Flask container (web) is successfully talking to the Redis container over the Compose network, and that Nginx is correctly proxying to it.

```
$ curl http://localhost:8080/
```

```
PS C:\flask-redis-nginx> curl http://localhost:8080/
>>
StatusCode      : 200
StatusDescription : OK
Content          : Hello! This page has been viewed 20 times.
                  Served by container: 22eccf8727c1

RawContent       : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 77
                  Content-Type: text/html; charset=utf-8
                  Date: Tue, 08 Sep 2026 09:54:30 GMT
                  Server: nginx/1.31.5

Forms            : {}
Headers          : {[Connection, keep-alive], [Content-Length, 77],
                  [Content-Type, text/html; charset=utf-8], [Date, Tue,
                  08 Sep 2026 09:54:30 GMT]...}
Images          : {}
InputFields      : {}
Links            : {}
ParsedHtml       : System.__ComObject
RawContentLength : 77

PS C:\flask-redis-nginx> curl http://localhost:8080/
>>
StatusCode      : 200
StatusDescription : OK
Content          : Hello! This page has been viewed 21 times.
                  Served by container: 22eccf8727c1

RawContent       : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 77
                  Content-Type: text/html; charset=utf-8
                  Date: Tue, 08 Sep 2026 09:54:58 GMT
                  Server: nginx/1.31.5

Forms            : {}
Headers          : {[Connection, keep-alive], [Content-Length, 77],
                  [Content-Type, text/html; charset=utf-8], [Date, Tue,
                  08 Sep 2026 09:54:58 GMT]...}
Images          : {}
InputFields      : {}
Links            : {}
ParsedHtml       : System.__ComObject
RawContentLength : 77
```

Step 11: Inspect the logs of an individual service (useful for debugging).

```
$ docker compose logs web
```

```
C:\flask-redis-nginx>docker compose logs web
web-1 | * Serving Flask app 'app'
web-1 | * Debug mode: off
web-1 | WARNING: This is a development server. Do not use it in a production dep
web-1 | * Running on all addresses (0.0.0.0)
web-1 | * Running on http://127.0.0.1:5000
web-1 | * Running on http://172.18.0.3:5000
web-1 | Press CTRL+C to quit
web-1 | 172.18.0.4 - - [08/Sep/2026 09:10:14] "GET / HTTP/1.1" 200 -
web-1 | 172.18.0.4 - - [08/Sep/2026 09:10:14] "GET /favicon.ico HTTP/1.1" 404 -
web-1 | 172.18.0.4 - - [08/Sep/2026 09:10:24] "GET / HTTP/1.1" 200 -
```

Step 12: Scale the web service to run multiple replicas. Because Nginx proxies to the service name "web" (not a fixed container), and Compose load-balances across replicas by default, this demonstrates horizontal scaling with zero code changes.

```
$ docker compose up -d --scale web=3
```

```
C:\flask-redis-nginx>docker compose up -d --scale web=3
[+] up 5/5
✓ Container flask-redis-nginx-redis-1 Running 0.0s ✓ Container flask-redis-nginx-web-1 Running 0.0s ✓ Container flask-redis-nginx-web-3 Started 0.2s ✓ Container flask-redis-nginx-web-2 Started 0.4s ✓ Container flask-redis-nginx-nginx-1 Running 0.0s
```

Step 13: Stop and remove all containers, and the network created for this project, once testing is complete.

```
$ docker compose down
```

```
C:\flask-redis-nginx>docker compose down
[+] down 6/6
✓ Container flask-redis-nginx-nginx-1 Removed
✓ Container flask-redis-nginx-web-1 Removed
✓ Container flask-redis-nginx-web-3 Removed
✓ Container flask-redis-nginx-web-2 Removed
✓ Container flask-redis-nginx-redis-1 Removed
✓ Network flask-redis-nginx_appnet Removed
```